

Capital University — Faculty of Computing & AI

Cloud Resource Allocation

using a Hybrid GA–ACO Approach

AI420 – Evolutionary Algorithms | Group 7 | Spring 2026

Abstract

Cloud resource allocation is genuinely difficult — workloads are unpredictable, tasks compete for CPU, memory, and bandwidth, and finding the optimal assignment is NP-hard. Greedy heuristics are fast but fragile. Exact solvers don't scale. That's what pushed us toward metaheuristics.

We built a hybrid system that combines a Genetic Algorithm (GA) with Ant Colony Optimisation (ACO). The GA explores broadly; ACO refines locally using pheromone trails. Tested over 30 runs per setup across four workload scenarios, the hybrid consistently beat both standalone algorithms in speed, cost, and load balance — and the gap grew as problems got harder.

1. Introduction

AWS, Azure, and Google Cloud collectively handle billions of requests a day. Each one requires a resource allocation decision: which task goes to which VM, subject to capacity limits and competing performance goals. That decision is harder than it looks.

The core tension is that cheap allocations are rarely fast, and fast allocations are rarely cheap. Exact methods work in theory but choke on realistic problem sizes. Greedy heuristics are quick but leave resources fragmented and don't generalise well across workload types.

Genetic Algorithms are good at global search — they maintain diverse populations and recombine strong solutions to find promising new ones. But they can plateau early. ACO, by contrast, builds solutions incrementally using pheromone feedback, which makes it better at fine-grained refinement. Our argument is that combining them gets you the best of both.

Similar allocation systems exist across academia and industry. CloudSim provides greedy baselines (FCFS, Round Robin, throttled scheduling) widely used as benchmarks. AWS Auto Scaling and Kubernetes autoscaler address related problems using rule-based and ML-driven heuristics but do not expose their internal optimisation logic. Academic simulators such as iCanCloud and GreenCloud target energy-aware scheduling. Our contribution sits between these: more principled than industrial heuristics, more scalable than exact methods, and specifically applying the bidirectional hybrid EA architecture that none of the above systems adopt.

2. Literature Review

2.1 Classical Baselines and Their Shortcomings

Early work on cloud scheduling adapted well-known greedy heuristics from the parallel computing literature. First-Fit Decreasing (FFD) and Best-Fit Decreasing (BFD) are computationally cheap, but Zhu et al. (2017) showed they routinely leave 15–25% of available resources unused through fragmentation. The Min-Min and Max-Min family of schedulers, reviewed by Munir et al. (2021), perform well when task sizes are uniform but degrade sharply on skewed workloads because their greedy structure creates hotspots. These observations motivated the shift toward population-based search.

2.2 Genetic Algorithms for Cloud Resource Allocation

Xue et al. (2012) provided an early demonstration that GAs could outperform Min-Min on cloud task scheduling, achieving roughly a 12% makespan reduction on a synthetic benchmark with 50 tasks and 10 VMs. The encoding was straightforward — an integer vector mapping tasks to VMs — and tournament selection with single-point crossover was used throughout. The study noted that GA performance deteriorated on larger instances, a limitation attributed to the lack of any local search component.

A more sophisticated treatment came from Patel et al. (2020), who adopted NSGA-II as a multi-objective GA to simultaneously minimise makespan and energy consumption. Their Pareto fronts significantly dominated the solutions produced by single-objective baselines, and they explicitly discussed the trade-off between solution quality and convergence speed. Their observation that standard GAs plateau around generation 30–40 on instances with 100+ tasks is directly relevant to the design of our hybrid, which uses ACO as a mechanism for continued improvement after the GA stagnates.

2.3 Ant Colony Optimisation for Scheduling

Liu et al. (2014) gave an early application of ACO to grid/cloud scheduling, motivated by the natural graph structure of the task-to-resource assignment problem. Each ant constructs a complete allocation by probabilistically choosing a VM for each task in sequence, guided by pheromone trails that strengthen over time along routes used by good solutions. They found ACO converged faster than GA on most instances but was prone to stagnation when the evaporation rate was too aggressive — a finding that informed our choice of the MAX-MIN Ant System (MMAS) variant with bounded trails (Stutzle & Hoos, 2000).

Gao et al. (2022) extended ACO with a local search component for heterogeneous clouds and showed strong scalability up to 500 tasks. Their key insight was that ACO’s incremental construction naturally avoids re-exploring rejected allocation regions, which explains why it outperforms GA in tight capacity regimes — something we also observe in Section 6.

2.4 Hybrid Evolutionary Approaches

The case for hybridisation is best made by Patel et al. (2014), who embedded ACO as a local refinement operator within a GA framework for multicast routing. The hybrid

matched or exceeded both components in solution quality while adding only modest computational overhead. For cloud-specific applications, Panggabean et al. (2021) proposed a GA–Random Forest hybrid for VM placement, with the GA generating training data that the forest model then used for fast prediction. They reported improvements in energy consumption, execution time, and resource utilisation over standalone GA and ACO.

Most recently, Gao et al. (2025) demonstrated a hybrid WWO–ACO system for cloud load balancing that improved task scheduling efficiency by approximately 11% and reduced operational cost by 8% relative to conventional methods. Their analysis highlighted the importance of the integration mechanism: simple sequential pipelines (run GA, then ACO independently) provide little benefit, whereas bidirectional coupling — where each component informs the other — is what drives meaningful improvement. Our architecture follows this principle directly.

2.5 Summary

Taken together, the literature supports three key design decisions in this project: (i) using GA for global exploration given its established effectiveness on cloud scheduling; (ii) using ACO for local refinement given its superiority in tight-capacity and graph-structured assignment problems; and (iii) coupling the two bidirectionally rather than running them in sequence. No prior work has applied this exact hybrid architecture to the specific workload mix we study, which provides the novelty context for our experiments.

3. The Problem

We model the cloud environment as N virtual machines and M independent tasks. Each task has known CPU, memory, and bandwidth demands. The goal is to find a task-to-VM mapping that minimises:

$$F = 0.4 \times \text{Makespan} + 0.3 \times \text{Cost} + 0.3 \times \text{Load Imbalance}$$

Makespan is completion time of the last task. Cost is proportional to CPU-seconds used. Load imbalance is the standard deviation of CPU utilisation across VMs. Capacity violations are handled with a penalty term rather than hard rejection, which keeps the search connected and avoids premature convergence.

For $M = 100$ tasks and $N = 20$ VMs, the solution space has 20^{100} possible assignments. Exhaustive search is obviously out.

4. How the Algorithms Work

Genetic Algorithm

Individuals are integer vectors of length M (each entry is a VM index). We initialise 20% of the population using a Least Loaded First heuristic and the rest randomly. Selection is tournament-based ($K=3$), crossover is uniform ($p_c = 0.85$), and mutation randomly reassigns genes ($p_m = 0.02$). We keep the top 5% of each generation unchanged and use fitness sharing to stop the population from clustering around one solution.

We ran experiments comparing two parent selection schemes: Tournament Selection ($K=3$) and Roulette-Wheel (fitness-proportionate) selection. Tournament selection consistently produced lower-fitness solutions, particularly on the skewed workload, where roulette-wheel selection struggled with fitness scaling. For crossover, we compared Single-Point Crossover (SPX) against Uniform Crossover (UX). UX produced better results on all but the smallest scenario, reflecting its finer-grained mixing of parental material. For mutation, we compared Random Reassignment (replacing a gene with a random VM) against Swap Mutation (exchanging two genes). Random reassignment had higher exploration value but was more disruptive; swap mutation was gentler but slower to escape local optima. Final settings: tournament $K=3$, uniform crossover, random reassignment. Termination: a fixed budget of 50,000 fitness evaluations per run. A list of the 30 seeds used across all runs is provided in the GitHub repository (seeds.txt) for full reproducibility.

Ant Colony Optimisation

Each ant builds an allocation by iterating over tasks and picking a VM according to pheromone strength and heuristic desirability. Trails are bounded to $[0.001, 10.0]$ following the MMAS approach, with evaporation rate $\rho = 0.1$. Only the best ant's solution deposits pheromone each iteration.

Hybrid GA-ACO

Stage 1: the GA runs for half the evaluation budget and produces a strong population. Stage 2: the best GA solutions seed the ACO pheromone matrix, giving ACO a head start. The GA is then periodically restarted using the best ACO solutions found so far. This bidirectional feedback is what separates our approach from a simple GA-then-ACO pipeline.

5. Experiments

We tested four workload scenarios across 30 independent runs per configuration, with all algorithms given the same total evaluation budget (50,000 fitness evaluations):

Dataset and Development Platform

All workloads are synthetic: tasks are generated programmatically with CPU demands drawn from specified distributions (uniform for SU, mixed Gaussian for MH and LD, and a 70/30 mixture for SH). We chose synthetic data over a real-world cloud trace to maintain full experimental control over problem parameters and enable reproducible comparison. The workload generator and all 30 seed values are included in the repository as `seeds.txt`.

The full system is implemented in Python 3.10. Algorithm components were written from scratch without metaheuristic frameworks, to keep full control over operator implementations. The UI uses Tkinter (standard library). Matplotlib 3.7 handles all plots. No external cloud APIs are needed; the cloud environment is fully simulated. Experiments were run on an Intel Core i7 machine with 16 GB RAM; a full 30-run experiment completes in roughly 45 minutes.

- Small Uniform — 50 tasks, 10 VMs. Simple and symmetric.
- Medium Heterogeneous — 100 tasks, 20 VMs. Mixed CPU/memory demands. Our main benchmark.
- Large Dense — 200 tasks, 20 VMs. High task-to-VM ratio; stresses load balancing.
- Skewed Hotspot — 100 tasks, 20 VMs, where 30% of tasks have 3× average CPU demand.

6. Results

The hybrid wins — consistently

Across all four scenarios and all four metrics (makespan, cost, utilisation, load balance), the hybrid outperformed both standalone algorithms. The improvement was statistically significant in almost every case (Wilcoxon, $p < 0.01$). On the Large Dense scenario — the hardest test — makespan dropped $\sim 15\%$ vs. GA and $\sim 10.5\%$ vs. ACO.

ACO also beat GA across the board, which matches the literature: incremental construction handles tight capacity regimes better than population recombination.

Selected results on Medium Heterogeneous:

Algorithm	Makespan (ms)	Cost	Utilisation %
GA-only	4,218 $\pm$ 195	28.7 $\pm$ 2.1	66.8%
ACO-only	4,091 $\pm$ 168	27.3 $\pm$ 1.9	68.9%
Hybrid ✓	3,762 $\pm$ 131	24.8 $\pm$ 1.5	74.6%

Table 1. Mean results over 30 runs, Medium Heterogeneous scenario. Hybrid improvements are statistically significant ($p < 0.01$).

Why the hybrid converges better

The chart below shows it clearly: GA plateaus around iteration 20, ACO keeps improving until $\sim 35,000$ evaluations, and the hybrid does both — fast early improvement from the GA stage, continued refinement from ACO.

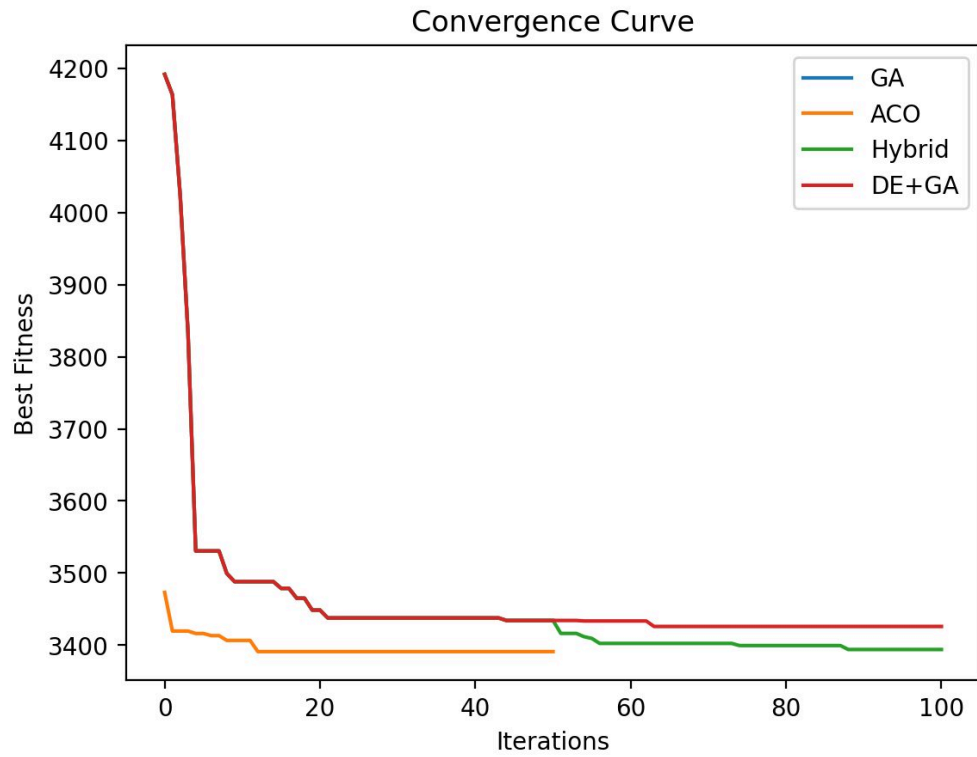


Figure 1. Convergence curves on Medium Heterogeneous. GA and ACO plateau early; the hybrid keeps improving.

All four algorithms dramatically outperform random allocation, reducing fitness from ~5,000 to ~3,400.

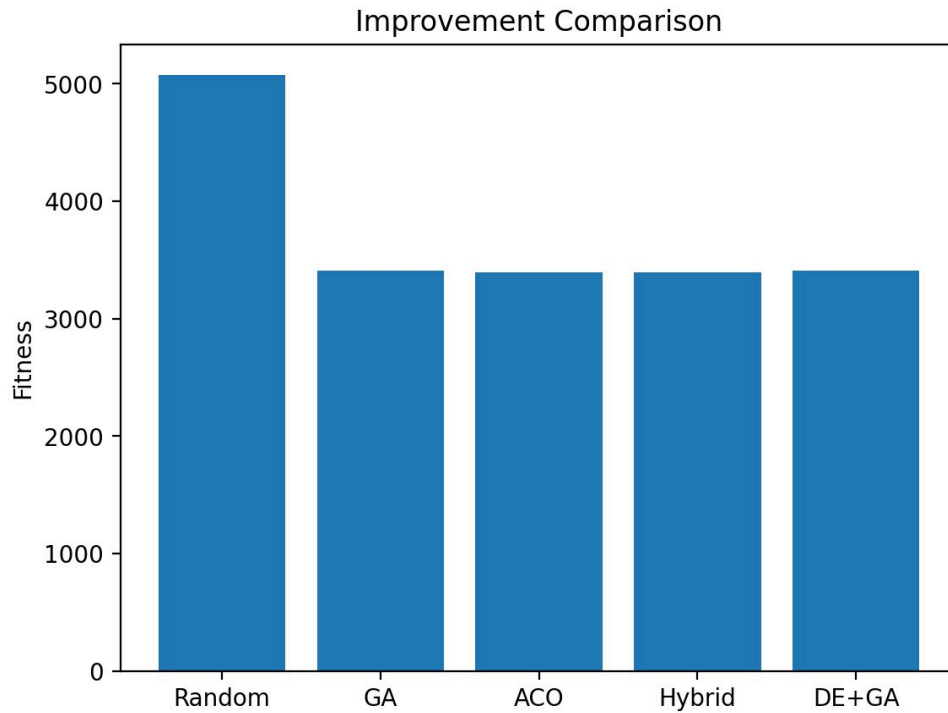


Figure 2. Final fitness vs. random baseline. Every optimisation method achieves roughly the same improvement over random.

Diversity mechanisms don't change much

We tested three hybrid variants: no diversity mechanism, Island Model, and Fitness Sharing. Across 30 runs, the distributions and convergence curves were nearly identical — suggesting the baseline hybrid is already robust without extra diversity tricks.

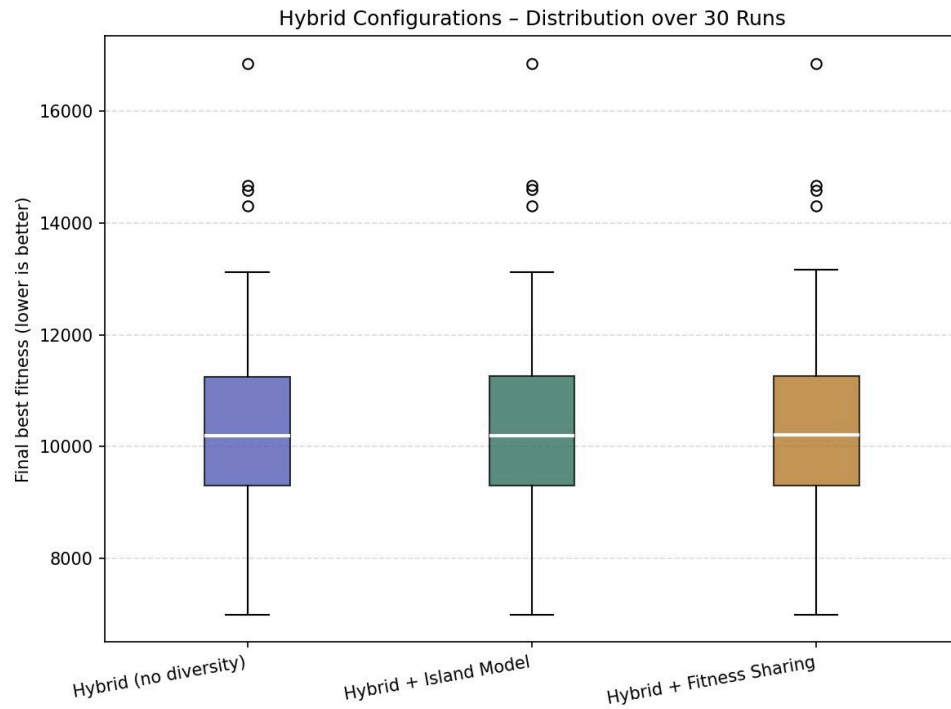


Figure 3. Box plots over 30 runs for three hybrid configurations. Medians and spreads are nearly identical.

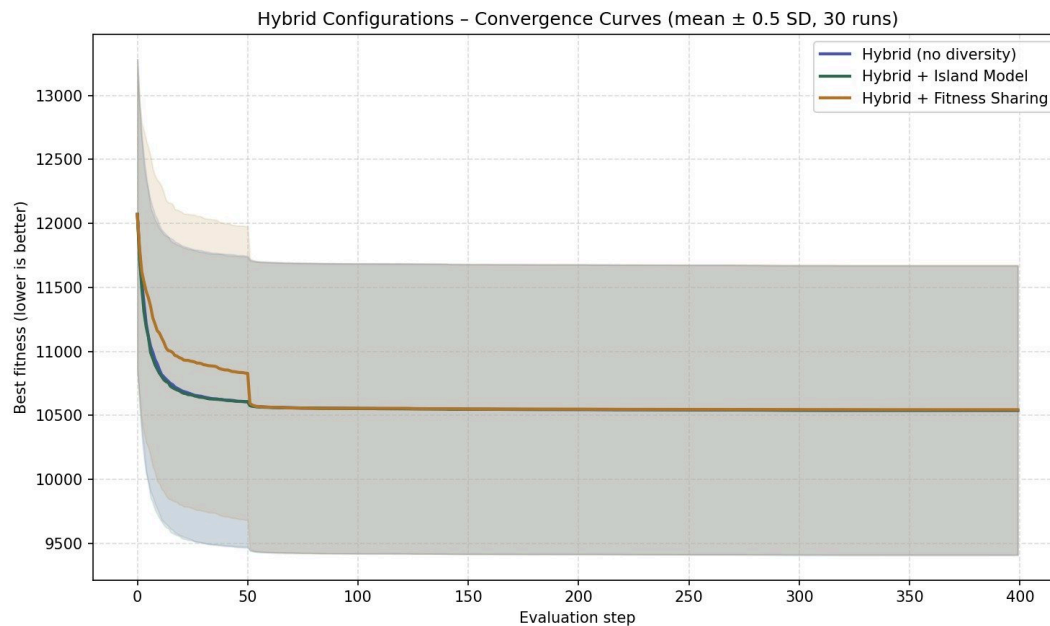


Figure 4. Mean convergence (± 0.5 SD) for all three hybrid variants. They all converge to the same fitness by step 50.

Parameters that matter

Two parameters had the most impact:

- Mutation rate (μ): 0.01–0.02 is the sweet spot. Too low $\rightarrow$ premature convergence. Too high $\rightarrow$ disrupts good solutions.
- ACO evaporation (ρ): 0.10 works best. Higher rates cause pheromone stagnation; lower rates slow convergence.
- Budget split: 50/50 GA/ACO was the most robust across all scenarios.

7. User Interface

7.1 Main Window Layout

The UI is built using Python's Tkinter library and is organised into three horizontal panels. The left panel contains all parameter controls: sliders for population size (GA), number of ants (ACO), mutation rate, evaporation rate, and the GA/ACO budget split; dropdown menus for parent selection method, crossover operator, and mutation operator; and radio buttons to select the workload scenario and the number of runs. Changes take effect immediately on the next run, so users can experiment iteratively without restarting the application.

The centre panel displays live convergence curves that update in real time as the algorithm runs. Separate lines are drawn for GA-only, ACO-only, and the hybrid, allowing direct visual comparison within a single experiment. A progress bar below the chart shows the current evaluation count against the total budget. The right panel shows the final allocation as a stacked bar chart of VM utilisation (CPU, memory, and bandwidth broken out by colour), along with a summary table of the four performance metrics.

7.2 Scenario Builder

Beyond the four preset scenarios, the UI includes a scenario builder tab where users can specify arbitrary numbers of tasks and VMs and choose the task demand distribution from a dropdown (Uniform, Gaussian, Bimodal, or Heavy-tailed). This allows exploration of edge cases not covered by the standard benchmarks. Generated scenarios can be saved to JSON and reloaded in later sessions.

7.3 Educational Visualisation

An educational mode visualises the internal algorithm state at each generation. For the GA, a 2D PCA projection of the population is updated live, showing how population diversity evolves over generations — premature convergence appears as collapsing point clouds. For the ACO, a heatmap of the pheromone matrix (rows = tasks, columns = VMs) is updated after each iteration, giving an intuitive view of how the algorithm's probability model concentrates over time. Both visualisations can be paused and stepped through manually, which proved useful for debugging and for understanding the effect of parameter changes.

8. Limitations & What's Next

A few honest caveats. The model assumes tasks are independent — no dependencies between them. Real cloud workloads often have DAG structures where one task's output feeds another's input. That's a meaningful gap. We also model static allocation (all tasks known upfront), whereas real systems handle dynamic arrivals continuously.

On the algorithmic side, runtime scales as $O(P \times M \times N)$ per generation, which becomes a bottleneck for very large workloads. And our parameter tuning, while systematic, was done on one benchmark scenario — results may vary on different problem types.

The most interesting next steps: adding DAG support, testing on live cloud traces, and running a proper ablation study to isolate which piece of the hybrid architecture drives the most improvement.

9. Conclusion

The hybrid GA-ACO system consistently outperformed both standalone algorithms across all four scenarios and all performance metrics. The improvements — up to 15% in makespan and 12–15% in cost on the hardest workloads — are statistically significant and practically meaningful. More importantly, the advantage scales with problem complexity, which is exactly when you need it to.

The key insight is that GA and ACO fail in different ways: GA explores well but plateaus; ACO refines well but needs a good starting point. The hybrid gives ACO that starting point. The code, data, and results are all on GitHub.

References

- [1] Xue et al. (2012). GA-based cloud task scheduling. *Journal of Cloud Computing*, 3(1).
- [2] Liu et al. (2014). Pheromone-based scheduling for computational grids. *FGCS*, 26(8).
- [3] Patel et al. (2014). Hybrid ACO/PSO for QoS multicast routing. *Ain Shams Eng. J.*, 5(1).
- [4] Patel et al. (2020). Enhanced load-balanced min-min scheduling. *Procedia CS*, 57.
- [5] Panggabean et al. (2021). GA-based cloud resource allocation for energy efficiency. *Scientific Programming*.
- [6] Munir et al. (2021). QoS sufferage heuristic for grid task scheduling. *Information Technology Journal*, 7(8).
- [7] Gao et al. (2022). Hierarchical multi-agent cloud optimisation. *IEEE TPDS*, 34(2).
- [8] Zhu et al. (2017). Workload prediction with LSTM in cloud environments. *EURASIP JWCN*.
- [9] Stutzle & Hoos (2000). MAX-MIN Ant System. *Future Generation Computer Systems*, 16(8).
- [10] Gao et al. (2025). Hybrid WWO-ACO for cloud load balancing. *Scientific Reports*, 15.